

# 第一部分：理论知识

## 1.1 什么是文件系统

### 学习目标

- 理解文件系统的概念和作用
- 了解 Linux 中"一切皆文件"的设计哲学

### 理论讲解

#### 什么是文件系统？

文件系统是操作系统用来组织和管理存储设备（硬盘、U盘、SSD等）上数据的一种方法。

### 生活化类比：图书馆管理系统

想象一下你走进一个大型图书馆：

| 图书馆            | Linux 文件系统 |
|----------------|------------|
| 整个图书馆          | 整个硬盘       |
| 书架区域（文学区、科技区等） | 目录/文件夹     |
| 每一本书           | 文件         |
| 图书编号系统         | 文件路径       |
| 图书管理员          | 操作系统       |
| 借书证权限          | 文件权限       |

如果图书馆没有管理系统，几百万本书就会乱成一团，你根本找不到想要的书。文件系统就是帮你在硬盘里"找到那本书"的管理系统。

# Linux 的核心哲学：一切皆文件

Linux 有一个非常重要的设计思想：一切皆文件（Everything is a file）。

这意味着在 Linux 中，不仅普通的文档、图片是文件，连以下这些都被当作"文件"来处理：

| 类型   | 举例                         | 说明           |
|------|----------------------------|--------------|
| 硬件设备 | <code>/dev/sda</code> （硬盘） | 硬盘被当作文件      |
| 键盘输入 | <code>/dev/input/</code>   | 你敲键盘产生的输入    |
| 进程信息 | <code>/proc/cpuinfo</code> | CPU 信息也是"文件" |
| 网络连接 | <code>/dev/tcp/</code>     | 网络也可以当文件读写   |

为什么要这样设计？

因为这样你只需要学会"读文件、写文件"这一套操作，就能操控整个系统！这就是 Linux 的优雅之处。

## 1.2 Linux 目录结构（FHS 标准）

### 学习目标

- 掌握 Linux 根目录下各主要目录的用途
- 理解 FHS（文件系统层级标准）

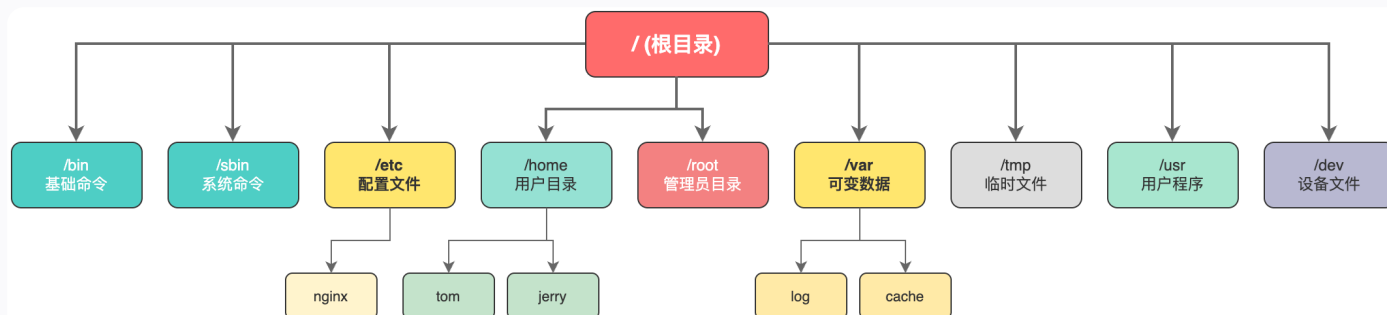
### 理论讲解

#### 生活化类比：一栋功能明确的大楼

把 Linux 系统想象成一栋大楼：

|      |            |                    |
|------|------------|--------------------|
| 顶楼   | /root      | → 老板办公室（管理员的家）     |
| 10F  | /home      | → 员工宿舍区（普通用户的家）    |
| 8F   | /var       | → 仓库（存放经常变化的数据）    |
| 6F   | /tmp       | → 临时会议室（临时文件，重启清空） |
| 4F   | /etc       | → 档案室（系统配置文件）      |
| 2F   | /usr       | → 工具间（用户安装的程序）     |
| 1F   | /bin /sbin | → 前台大厅（基础命令工具）     |
| 地下室  | /dev       | → 机房（硬件设备）         |
| 大楼入口 | /          | → 根目录（一切的起点）       |

## 核心目录详解



### 图例

运维重点关注

| 目录    | 英文全称                  | 用途说明                   | 运维场景      |
|-------|-----------------------|------------------------|-----------|
| /     | root                  | 根目录，所有目录的起点            | 整个系统的入口   |
| /bin  | binary                | 基础命令（ls, cp, cat等）     | 系统修复时会用到  |
| /sbin | system binary         | 系统管理命令（reboot, fdisk等） | 管理员专用命令   |
| /etc  | etcetera              | 系统配置文件                 | 运维最常访问的目录 |
| /home | home                  | 普通用户的家目录               | 用户数据存放处   |
| /root | root                  | 管理员的家目录                | root 用户专属 |
| /var  | variable              | 可变数据（日志、缓存等）           | 查日志必去的地方  |
| /tmp  | temporary             | 临时文件                   | 存放临时数据    |
| /usr  | Unix System Resources | 用户安装的程序和文件             | 软件安装目录    |
| /dev  | device                | 设备文件                   | 硬盘、终端等设备  |
| /proc | process               | 虚拟文件系统，进程和内核信息         | 查看系统运行状态  |
| /opt  | optional              | 第三方软件安装目录              | 手动安装软件常用  |

 运维小贴士

作为运维工程师，你最常打交道的目录是：

1. `/etc` — 改配置

2. `/var/log` — 查日志
  3. `/home` — 管用户数据
  4. `/tmp` — 临时处理文件
- 

## 1.3 文件类型与权限基础

### 学习目标

- 识别 Linux 中的 7 种文件类型
- 初步理解权限的概念

### 理论讲解

#### Linux 文件类型

当你使用 `ls -l` 命令时，第一个字符就代表文件类型：

```
-rw-r--r--  1 root root  1234 Dec 10 10:00 readme.txt
drwxr-xr-x  2 root root  4096 Dec 10 10:00 mydir
lrwxrwxrwx  1 root root    9 Dec 10 10:00 link -> readme.txt
```



# 1.4 绝对路径与相对路径

## 🎯 学习目标

- 区分绝对路径和相对路径
- 掌握路径的书写方法

## 📖 理论讲解

### 🗺️ 生活化类比：导航方式

假设你要告诉外卖小哥你家地址：

| 方式   | 导航例子                   | Linux 路径                                    |
|------|------------------------|---------------------------------------------|
| 绝对地址 | "北京市朝阳区望京街道XX小区3号楼201" | <code>/home/tom/documents/report.txt</code> |
| 相对地址 | "从你现在的位置往前走100米右转"     | <code>./documents/report.txt</code>         |

### 绝对路径

- 特点：从根目录 `/` 开始，写出完整路径
- 优点：无论你在哪个目录，都能准确定位
- 格式： `/目录1/目录2/文件名`

```
# 绝对路径示例
/home/tom/documents/report.txt
/etc/nginx/nginx.conf
/var/log/messages
```

### 相对路径

- 特点：从当前目录出发，描述相对位置
- 优点：书写简短
- 常用符号：

| 符号 | 含义       | 类比      |
|----|----------|---------|
| .  | 当前目录     | "我在这里"  |
| .. | 上一级目录    | "往回走一层" |
| ~  | 当前用户的家目录 | "回家"    |

# 假设当前在 /home/tom 目录

./documents/report.txt

# 当前目录下的 documents/report.txt

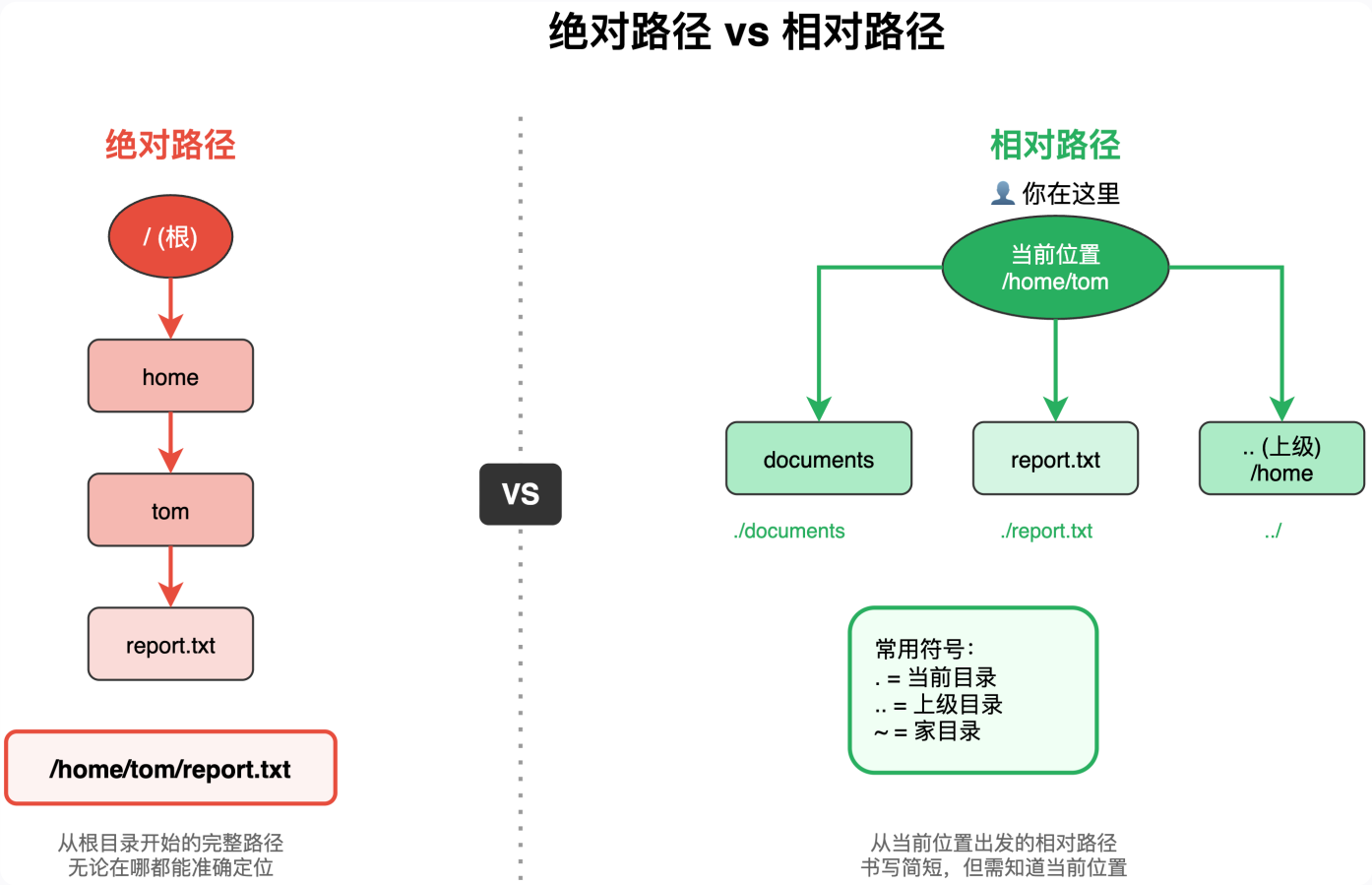
../jerry/photo.jpg

# 上一级目录下的 jerry/photo.jpg

~/downloads/

# 家目录下的 downloads (等于

/home/tom/downloads)





## 1.5 \$PATH的作用

关于可执行文件路径的变量： \$PATH

```
$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/root/bin

$ export PATH=$PATH:/root
```

当我们在执行一个指令的时候，举例来说“ls”好了，系统会依照PATH的设置去每个PATH定义的目录下搜寻文件名为ls的可执行文件，如果在PATH定义的目录中含有多个文件名为ls的可执行文件，那么先搜寻到的同名指令先被执行！

课堂实验：

例题：假设你是root，如果你将ls由/bin/ls移动成为/root/ls，然后你自己本身也在/root目录下，

请问：

1. 你能不能直接输入ls来执行？
2. 若不能，你该如何执行ls这个指令？
3. 若要直接输入ls即可执行，又该如何进行？

## 第二部分： 实践操作

### 2.1 目录浏览与切换

#### 学习目标

- 掌握查看当前位置、切换目录、列出内容的操作

#### 实践命令

## pwd — 查看当前位置 (Print Working Directory)

```
# 查看当前所在目录
$ pwd
/home/tom
```

 类比：你问导航"我现在在哪里？"

## cd — 切换目录 (Change Directory)

```
# 进入指定目录 (绝对路径)
$ cd /var/log

# 进入指定目录 (相对路径)
$ cd documents

# 返回上一级目录
$ cd ..

# 返回上两级目录
$ cd ../..

# 回到家目录 (三种方式都可以)
$ cd
$ cd ~
$ cd $HOME

# 返回上一次所在的目录
$ cd -
```

 类比：就像在大楼里走动——去几楼、上楼、下楼、回家。

## ls — 列出目录内容 (List)

```
# 列出当前目录内容
$ ls
Desktop  Documents  Downloads  Music  Pictures
```

```
# 列出详细信息
$ ls -l
total 20
drwxr-xr-x 2 tom tom 4096 Dec 10 10:00 Desktop
drwxr-xr-x 2 tom tom 4096 Dec 10 10:00 Documents
-rw-r--r-- 1 tom tom 123 Dec 10 10:00 readme.txt

# 显示隐藏文件（以 . 开头的文件）
$ ls -a
.  ..  .bashrc  .profile  Desktop  Documents

# 组合使用：详细信息 + 隐藏文件 + 人类可读大小
$ ls -lah
total 32K
drwxr-xr-x 5 tom tom 4.0K Dec 10 10:00 .
drwxr-xr-x 3 root root 4.0K Dec 10 09:00 ..
-rw-r--r-- 1 tom tom 220 Dec 10 09:00 .bashrc
drwxr-xr-x 2 tom tom 4.0K Dec 10 10:00 Desktop

# 列出指定目录的内容
$ ls -l /etc
```

#### 常用 ls 选项速查：

| 选项              | 作用               |
|-----------------|------------------|
| <code>-l</code> | 详细列表格式           |
| <code>-a</code> | 显示所有文件（包括隐藏文件）   |
| <code>-h</code> | 人类可读的文件大小（KB、MB） |
| <code>-t</code> | 按修改时间排序          |
| <code>-r</code> | 逆序排列             |
| <code>-R</code> | 递归列出子目录          |
| <code>-S</code> | 按文件大小排序          |

## 动手练习

# 练习1: 探索系统目录

|                     |             |
|---------------------|-------------|
| <code>cd /</code>   | # 进入根目录     |
| <code>ls</code>     | # 查看根目录下有什么 |
| <code>cd etc</code> | # 进入 etc 目录 |
| <code>pwd</code>    | # 确认当前位置    |
| <code>ls -la</code> | # 查看详细内容    |

# 练习2: 回家再出门

|                          |                 |
|--------------------------|-----------------|
| <code>cd ~</code>        | # 回到家目录         |
| <code>pwd</code>         | # 确认位置          |
| <code>cd /var/log</code> | # 去日志目录         |
| <code>cd -</code>        | # 返回刚才的位置 (家目录) |

## 2.2 文件与目录的创建

### 学习目标

- 掌握创建空文件和目录的方法

### 实践命令

**touch** — 创建空文件 / 更新时间戳

```
# 创建单个空文件
$ touch myfile.txt

# 创建多个文件
$ touch file1.txt file2.txt file3.txt

# 批量创建带编号的文件
$ touch test{1..5}.txt
# 结果: test1.txt test2.txt test3.txt test4.txt test5.txt

# 如果文件已存在, touch 会更新它的修改时间
$ touch existing_file.txt
```

👉 类比: touch 就像"触碰"一下文件。文件不存在就创建, 存在就更新时间戳。

## mkdir — 创建目录 (Make Directory)

```
# 创建单个目录
$ mkdir mydir

# 创建多个目录
$ mkdir dir1 dir2 dir3

# 创建多级目录 (父目录不存在时自动创建)
$ mkdir -p projects/2024/december
# 一次性创建三层目录: projects → 2024 → december

# 创建目录并设置权限
$ mkdir -m 755 scripts
```

📁 类比: mkdir 就像买新文件夹, `-p` 选项就像一次性买好几层嵌套的文件夹。

## 👉 动手练习

```
# 练习: 创建一个项目目录结构
cd ~
mkdir -p myproject/{src,docs,tests,logs}
```

```
# 查看创建结果
ls -la myproject/
# 结果: src docs tests logs

# 在各目录下创建文件
touch myproject/src/main.py
touch myproject/docs/readme.md
touch myproject/logs/app.log

# 查看整体结构
ls -lR myproject/
```

## 2.3 文件与目录的复制、移动、重命名

### 学习目标

- 掌握文件和目录的复制、移动、重命名操作

### 实践命令

#### cp — 复制 (Copy)

```
# 复制文件
$ cp source.txt destination.txt

# 复制文件到另一个目录
$ cp myfile.txt /tmp/

# 复制并重命名
$ cp myfile.txt /tmp/newname.txt

# 复制目录 (必须加 -r 递归复制)
$ cp -r mydir/ /backup/

# 复制时保留权限、时间戳等属性
$ cp -p important.conf important.conf.bak
```

```
# 复制时显示过程
$ cp -v largefile.zip /backup/
'largefile.zip' -> '/backup/largefile.zip'

# 交互式复制（目标存在时询问是否覆盖）
$ cp -i source.txt dest.txt
cp: overwrite 'dest.txt'? y
```

 **cp 常用选项：**

| 选项 | 作用             |
|----|----------------|
| -r | 递归复制目录         |
| -p | 保留文件属性         |
| -v | 显示复制过程         |
| -i | 覆盖前询问          |
| -f | 强制覆盖           |
| -a | 归档模式（相当于 -dpr） |

## mv — 移动 / 重命名 (Move)

```
# 移动文件
$ mv file.txt /tmp/

# 移动多个文件到目录
$ mv file1.txt file2.txt file3.txt /backup/

# 重命名文件
$ mv oldname.txt newname.txt

# 重命名目录
$ mv old_folder new_folder
```

```
# 移动并重命名
$ mv /home/tom/report.txt /tmp/report_backup.txt

# 交互式移动
$ mv -i source.txt dest.txt

# 移动时显示过程
$ mv -v file.txt /tmp/
renamed 'file.txt' -> '/tmp/file.txt'
```

🚚 类比：cp 是复印，mv 是搬家。重命名其实就是"原地搬家换门牌号"。

## 👉 动手练习

```
# 练习：备份与整理文件

# 1. 创建测试环境
cd ~
mkdir -p practice/{original,backup}
touch practice/original/{a,b,c}.txt

# 2. 复制整个目录作为备份
cp -r practice/original practice/backup/original_backup

# 3. 重命名文件
cd practice/original
mv a.txt apple.txt
mv b.txt banana.txt

# 4. 移动文件
mv c.txt ../backup/

# 5. 查看结果
ls -lR ~/practice/
```

## 2.4 文件与目录的删除



## 学习目标

- 掌握安全删除文件和目录的方法
- 理解危险操作的防范

## 实践命令

### rm — 删除文件 (Remove)

```
# 删除单个文件
$ rm file.txt

# 删除多个文件
$ rm file1.txt file2.txt file3.txt

# 交互式删除 (推荐, 会询问确认)
$ rm -i important.txt
rm: remove regular file 'important.txt'? y

# 强制删除 (不询问)
$ rm -f file.txt

# 删除目录及其内容 (递归删除)
$ rm -r mydir/

# 递归强制删除 (危险!)
$ rm -rf mydir/

# 显示删除过程
$ rm -v file.txt
removed 'file.txt'
```

### rmdir — 删除空目录

```
# 删除空目录
$ rmdir emptydir

# 如果目录不为空，会报错
$ rmdir nonemptydir
rmdir: failed to remove 'nonemptydir': Directory not empty
```

## ⚠ 危险警告

```
# ❌ 绝对不要执行以下命令!!!
$ rm -rf /           # 删除整个系统
$ rm -rf /*          # 删除根目录下所有内容
$ rm -rf ~/./*        # 删除家目录所有内容
$ rm -rf ./.*         # 删除当前目录所有内容（确认你知道自己在哪）
```

## 🛡 安全建议

```
# 建议1：删除前先用 ls 确认
$ ls mydir/
$ rm -r mydir/

# 建议2：使用 -i 交互模式
$ rm -ri mydir/

# 建议3：设置别名，让 rm 默认询问
$ alias rm='rm -i'

# 建议4：使用 trash-cli 代替 rm（先安装）
$ trash-put file.txt      # 放入回收站
$ trash-list              # 查看回收站
$ trash-restore            # 恢复文件
```

## 👉 动手练习

```
# 练习：安全删除操作

# 1. 创建测试文件和目录
cd ~
mkdir -p delete_test/{dir1,dir2}
touch delete_test/file{1..3}.txt
touch delete_test/dir1/inside.txt

# 2. 尝试用 rmdir 删除非空目录（会失败）
rmdir delete_test/dir1
# 输出: rmdir: failed to remove 'delete_test/dir1': Directory not
empty

# 3. 删除空目录
rmdir delete_test/dir2

# 4. 交互式删除文件
rm -i delete_test/file1.txt

# 5. 递归删除目录
rm -ri delete_test/dir1

# 6. 清理
rm -r delete_test/
```

## 2.5 文件内容查看

### 学习目标

- 掌握多种查看文件内容的方法
- 根据场景选择合适的命令

### 实践命令

#### cat — 查看完整内容（Concatenate）

```
# 查看文件内容
```

```
$ cat file.txt

# 显示行号
$ cat -n file.txt
    1 This is line 1
    2 This is line 2
    3 This is line 3

# 查看多个文件
$ cat file1.txt file2.txt

# 合并文件内容到新文件
$ cat file1.txt file2.txt > merged.txt
```

 **适用场景：**小文件，需要看全部内容。

## head — 查看开头内容

```
# 默认显示前 10 行
$ head file.txt

# 显示前 5 行
$ head -n 5 file.txt
$ head -5 file.txt          # 简写

# 显示前 100 个字节
$ head -c 100 file.txt
```

 **适用场景：**快速预览文件开头，确认文件格式。

## tail — 查看结尾内容

```
# 默认显示最后 10 行
$ tail file.txt

# 显示最后 20 行
$ tail -n 20 file.txt
$ tail -20 file.txt          # 简写

# 🔥 实时跟踪文件变化（运维神器！）
$ tail -f /var/log/messages

# 从第 5 行开始显示到末尾
$ tail -n +5 file.txt
```

← **适用场景：**查看日志最新内容，实时监控日志。

## less — 分页查看（推荐用于大文件）

```
$ less largefile.log
```

**less 内部操作快捷键：**

| 按键     | 作用        |
|--------|-----------|
| 空格 / f | 向下翻一页     |
| b      | 向上翻一页     |
| ↓ / j  | 向下一行      |
| ↑ / k  | 向上一行      |
| g      | 跳到文件开头    |
| G      | 跳到文件末尾    |
| /关键词   | 向下搜索      |
| ?关键词   | 向上搜索      |
| n      | 跳到下一个搜索结果 |
| N      | 跳到上一个搜索结果 |
| q      | 退出        |

 类比：less 就像一个电子书阅读器，可以翻页、搜索、跳转。

## more — 分页查看（较老的命令）

```
$ more file.txt
# 空格翻页，q 退出
```

## 命令选择指南

| 场景      | 推荐命令    |
|---------|---------|
| 小文件，看全部 | cat     |
| 看文件开头几行 | head    |
| 看文件最后几行 | tail    |
| 实时监控日志  | tail -f |
| 大文件分页查看 | less    |

## 动手练习

```
# 练习：查看系统日志

# 1. 查看密码文件前 5 行
head -5 /etc/passwd

# 2. 查看系统日志最后 20 行
sudo tail -20 /var/log/syslog      # Ubuntu/Debian
sudo tail -20 /var/log/messages   # CentOS/RHEL

# 3. 用 less 浏览大文件
less /etc/services
# 尝试：空格翻页、/ssh 搜索、q 退出

# 4. 实时监控日志（新开终端触发一些操作）
sudo tail -f /var/log/syslog
# Ctrl+C 退出
```

## 2.6 文件查找

### 学习目标

- 掌握三种常用的文件查找命令

- 理解它们的区别和适用场景

## 实践命令

### find — 实时搜索（最强大）

```
# 基本语法
$ find 搜索路径 搜索条件 处理动作

# 按名称查找
$ find /home -name "*.txt"           # 在 /home 下找所有 .txt 文件
$ find . -name "config*"            # 当前目录下找 config 开头的文件
$ find / -name "nginx.conf" 2>/dev/null # 全盘搜索，忽略权限错误

# 不区分大小写
$ find . -iname "readme*"

# 按类型查找
$ find /var -type f                 # 只找文件
$ find /home -type d               # 只找目录
$ find /dev -type l                # 只找链接

# 按大小查找
$ find / -size +100M                # 大于 100MB 的文件
$ find /tmp -size -1k               # 小于 1KB 的文件
$ find /var -size 10M               # 正好 10MB 的文件

# 按时间查找
$ find /var/log -mtime -7           # 7 天内修改过的文件
$ find /tmp -atime +30              # 30 天前访问过的文件
$ find . -mmin -60                  # 60 分钟内修改过的文件

# 按权限查找
$ find / -perm 777                  # 权限为 777 的文件

# 找到后执行操作
$ find /tmp -name "*.log" -delete   # 找到并删除
$ find . -name "*.sh" -exec chmod +x {} \; # 找到并添加执行权限
$ find /var/log -name "*.log" -exec ls -lh {} \; # 找到并显示详情
```



 **find** 就像派出一支搜索队，实时搜遍每个角落，功能强大但速度较慢。

## locate — 数据库搜索（速度快）

```
# 快速搜索文件
$ locate nginx.conf
/etc/nginx/nginx.conf

# 不区分大小写
$ locate -i README

# 显示找到的数量
$ locate -c "*.log"

# 更新数据库（新文件需要先更新才能找到）
$ sudo updatedb
```

 **locate** 就像用索引查字典，速度快但数据可能不是最新的。

## which / whereis — 查找命令位置

```
# which: 查找命令的可执行文件路径
$ which python
/usr/bin/python

$ which ls
/usr/bin/ls

# whereis: 查找命令的程序、源码、手册位置
$ whereis nginx
nginx: /usr/sbin/nginx /usr/lib/nginx /etc/nginx /usr/share/nginx
/usr/share/man/man8/nginx.8.gz
```

## 查找命令对比

| 命令                  | 速度 | 实时性   | 功能   | 适用场景      |
|---------------------|----|-------|------|-----------|
| <code>find</code>   | 慢  | 实时    | 最强大  | 精确查找、复杂条件 |
| <code>locate</code> | 快  | 依赖数据库 | 简单   | 快速模糊查找    |
| <code>which</code>  | 快  | 实时    | 只找命令 | 查命令路径     |

## 动手练习

```
# 练习1: 用 find 搜索
# 找出 /etc 下所有 .conf 配置文件
find /etc -name "*.conf" 2>/dev/null | head -20

# 找出 7 天内修改过的日志文件
sudo find /var/log -name "*.log" -mtime -7

# 找出大于 50MB 的文件
find / -size +50M 2>/dev/null

# 练习2: 用 locate 快速搜索
sudo updatedb                # 先更新数据库
locate nginx
locate "*.service" | head -10

# 练习3: 查找命令位置
which bash
whereis python
```

## 2.7 链接文件

### 学习目标

- 理解 inode 和 block 的基本概念
- 理解硬链接和软链接的区别
- 掌握创建链接的方法

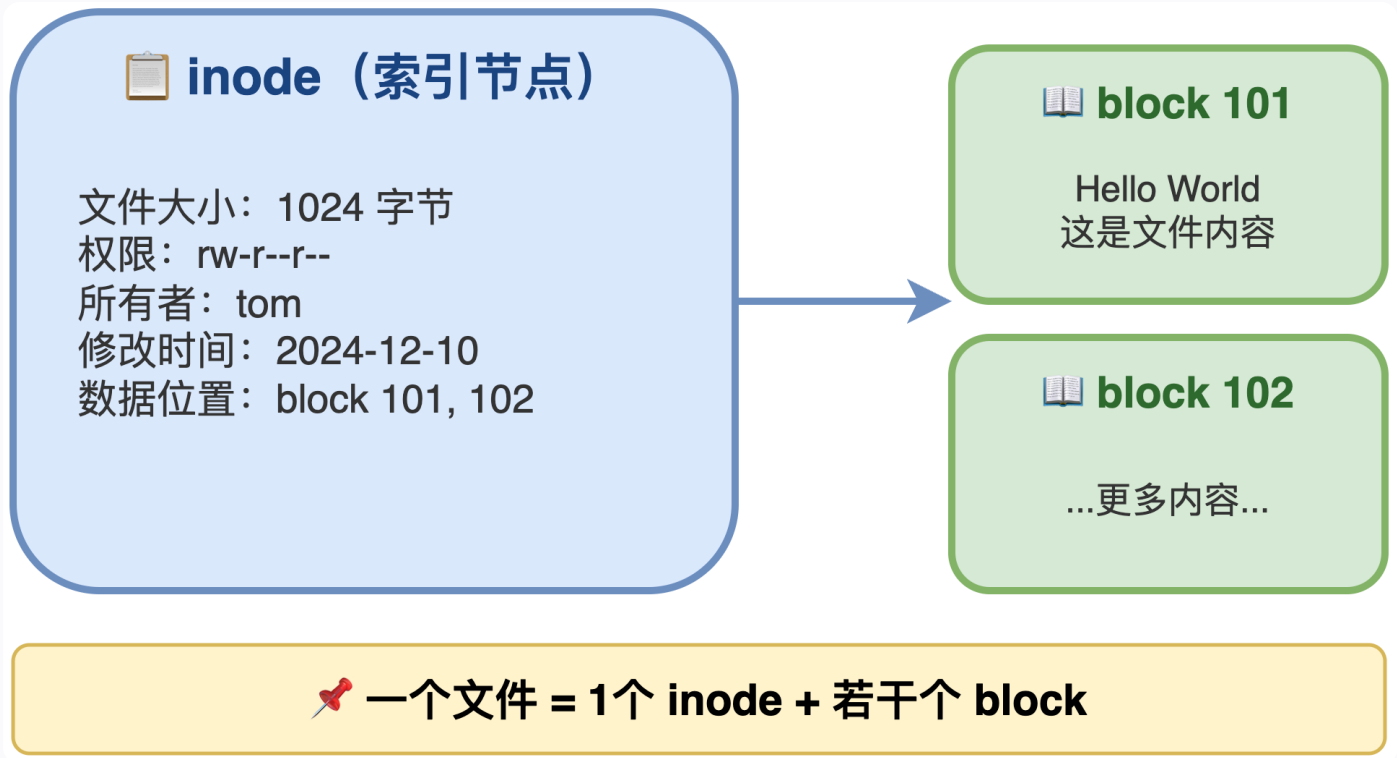
# 📖 理论讲解

## 👉 先理解两个概念：inode 和 block

在讲链接之前，我们需要先了解 Linux 是怎么存储文件的。

生活化类比：图书馆的书

| 概念           | 类比       | 说明                     |
|--------------|----------|------------------------|
| block (块)    | 📖 书的内容页  | 真正存放数据的地方（文字、图片等）      |
| inode (索引节点) | 📄 图书馆索引卡 | 记录文件的"身份信息"（大小、权限、位置等） |



简单记忆：

- inode = 文件的"身份证"（记录文件信息，但不包含文件名！）
- block = 文件的"内容本身"
- 文件名 = 只是 inode 的一个"别名"，存在目录里

💡 **重点：**文件名不存在 inode 里！文件名存在目录中，目录记录了"文件名 → inode 编号"的对应关系。

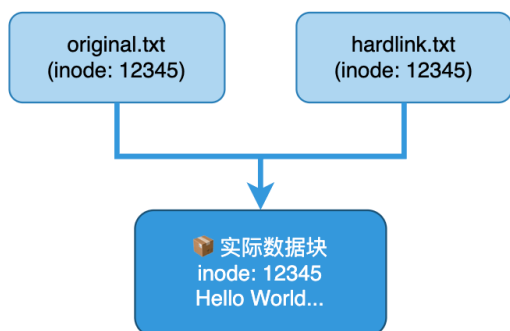
## 🔗 硬链接和软链接的本质区别

理解了 inode 和 block，链接就很好懂了：

| 类型  | 本质                       | 类比              |
|-----|--------------------------|-----------------|
| 硬链接 | 给同一个 inode 再起一个名字        | 一个人有两个名字（大名、小名） |
| 软链接 | 创建一个新文件，内容是"指向另一个文件名的路径" | 快捷方式 / 门牌号      |

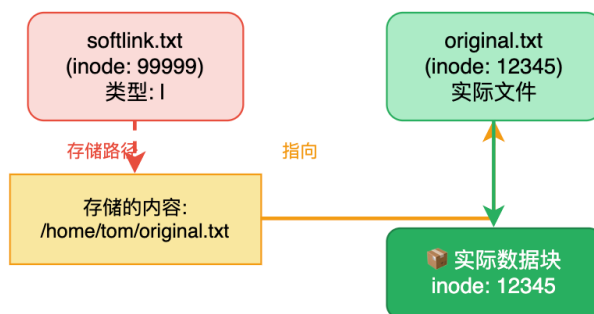
## 硬链接 vs 软链接 原理对比

### 硬链接 (Hard Link)



- ✅ 两个文件名指向同一份数据
- ✅ inode 编号相同
- ✅ 删除其中一个，另一个仍可用
- ❌ 不能跨文件系统
- ❌ 不能链接目录

### 软链接 (Symbolic Link)



- ✅ 可以跨文件系统
- ✅ 可以链接目录
- ✅ 链接文件很小（只存路径）
- ❌ 原文件删除后链接失效
- ❌ inode 与原文件不同

### 💡 生活化类比

硬链接 = 一个人有两个名字（大名和小名都是他自己）

软链接 = 快捷方式 / 门牌号（指向真正的位置）

| 特性      | 硬链接     | 软链接    |
|---------|---------|--------|
| 能否跨文件系统 | ✗ 不能    | ✓ 可以   |
| 能否链接目录  | ✗ 不能    | ✓ 可以   |
| 原文件删除后  | ✓ 链接仍可用 | ✗ 链接失效 |
| inode   | 相同      | 不同     |
| 显示方式    | 普通文件    | l 开头   |

## 实践命令

### ln — 创建链接 (Link)

```
# 创建硬链接
$ ln 源文件 链接名
$ ln original.txt hardlink.txt

# 创建软链接（符号链接）-- 加 -s 选项
$ ln -s 源文件或目录 链接名
$ ln -s /etc/nginx/nginx.conf ~/nginx.conf
$ ln -s /var/log ~/logs

# 查看链接指向
$ ls -l
lrwxrwxrwx 1 tom tom 21 Dec 10 10:00 nginx.conf ->
/etc/nginx/nginx.conf
lrwxrwxrwx 1 tom tom 8 Dec 10 10:00 logs -> /var/log

# 查看 inode（硬链接 inode 相同）
$ ls -li original.txt hardlink.txt
131074 -rw-r--r-- 2 tom tom 100 Dec 10 10:00 hardlink.txt
131074 -rw-r--r-- 2 tom tom 100 Dec 10 10:00 original.txt
```

## 运维常见用法

```
# 场景1: 为配置文件创建快捷方式
$ ln -s /etc/nginx/nginx.conf ~/nginx.conf
# 这样在家目录就能直接编辑 nginx 配置

# 场景2: 为长路径创建短别名
$ ln -s /var/www/html/myproject/current ~/project

# 场景3: 版本切换
$ ln -s /opt/jdk-11 /opt/java      # java 指向 jdk-11
$ rm /opt/java
$ ln -s /opt/jdk-17 /opt/java      # 切换到 jdk-17
```

## 动手练习

```
# 练习: 理解 inode 和链接

cd ~
mkdir link_test && cd link_test

# 1. 创建原始文件
echo "Hello, this is original content" > original.txt

# 2. 查看原文件的 inode
ls -li original.txt
# 输出类似: 131074 -rw-r--r-- 1 tom tom 33 Dec 10 10:00 original.txt
#           ↑ inode           ↑ 链接数=1

# 3. 创建硬链接
ln original.txt hardlink.txt

# 4. 再次查看 inode
ls -li
# 131074 -rw-r--r-- 2 tom tom 33 Dec 10 10:00 hardlink.txt
# 131074 -rw-r--r-- 2 tom tom 33 Dec 10 10:00 original.txt
# ↑ inode 相同!           ↑ 链接数变成 2

# 5. 创建软链接
ln -s original.txt softlink.txt
```

```
# 6. 查看软链接的 inode
ls -li softlink.txt
# 131099 lrwxrwxrwx 1 tom tom 12 Dec 10 10:00 softlink.txt ->
original.txt
# ↑ inode 不同!

# 7. 删除原文件, 观察区别
rm original.txt
cat hardlink.txt      # ✅ 硬链接仍可用 (数据还在)
cat softlink.txt      # ❌ 软链接失效! 报错 "No such file or directory"

# 8. 清理
cd ~ && rm -r link_test
```

## 2.8 文件打包与压缩

### 学习目标

- 理解打包与压缩的区别
- 掌握 tar 和 zip 的常用操作

### 理论讲解

#### 打包 vs 压缩

| 操作 | 作用           | 类比        |
|----|--------------|-----------|
| 打包 | 把多个文件合并成一个文件 | 把衣服装进行李箱  |
| 压缩 | 减小文件体积       | 用真空袋把衣服压扁 |

Linux 中通常先打包再压缩, 所以常见 `.tar.gz` 这样的双后缀。

#### 常见格式

| 格式                                       | 说明         | 使用场景          |
|------------------------------------------|------------|---------------|
| <code>.tar.gz</code> / <code>.tgz</code> | 最常用，压缩速度快  | 日常备份、传输       |
| <code>.tar.bz2</code>                    | 压缩率更高，速度较慢 | 需要更小体积        |
| <code>.zip</code>                        | 跨平台兼容      | 发给 Windows 用户 |

## 实践命令

### tar — 打包压缩神器

选项速记口诀

|                |                |                      |           |
|----------------|----------------|----------------------|-----------|
| <code>c</code> | <code>=</code> | <code>Create</code>  | 创建        |
| <code>x</code> | <code>=</code> | <code>eXtract</code> | 解压        |
| <code>t</code> | <code>=</code> | <code>lisT</code>    | 查看        |
| <code>v</code> | <code>=</code> | <code>Verbose</code> | 显示过程      |
| <code>f</code> | <code>=</code> | <code>File</code>    | 指定文件名     |
| <code>z</code> | <code>=</code> | <code>gzip</code>    | 用 gzip 压缩 |

### 创建压缩包

|                                                                                   |
|-----------------------------------------------------------------------------------|
| <code># 打包并压缩目录（最常用）</code>                                                       |
| <code>tar -czvf backup.tar.gz /path/to/dir/</code>                                |
| <code># 打包多个文件</code>                                                             |
| <code>tar -czvf files.tar.gz file1.txt file2.txt dir1/</code>                     |
| <code># 排除某些文件（运维常用）</code>                                                       |
| <code>tar -czvf backup.tar.gz --exclude="*.log" --exclude=".git" /var/www/</code> |

### 解压压缩包



```
# 解压到当前目录
tar -xzvf backup.tar.gz

# 解压到指定目录
tar -xzvf backup.tar.gz -C /opt/restore/
```

## 查看压缩包内容（不解压）

```
tar -tvf backup.tar.gz
```

## zip / unzip — 跨平台压缩

```
# 压缩目录
zip -r archive.zip mydir/

# 解压
unzip archive.zip

# 解压到指定目录
unzip archive.zip -d /opt/

# 查看内容
unzip -l archive.zip
```

---

## 命令速查

| 操作         | 命令                                           |
|------------|----------------------------------------------|
| 创建 .tar.gz | <code>tar -czvf name.tar.gz files/</code>    |
| 解压 .tar.gz | <code>tar -xzvf name.tar.gz</code>           |
| 解压到指定目录    | <code>tar -xzvf name.tar.gz -C /path/</code> |
| 查看内容       | <code>tar -tvf name.tar.gz</code>            |
| 创建 .zip    | <code>zip -r name.zip files/</code>          |
| 解压 .zip    | <code>unzip name.zip</code>                  |

## 运维实战场景

### 场景一：备份网站目录

```
# 带日期的备份
tar -czvf /backup/www_$(date +%Y%m%d).tar.gz /var/www/html/
```

### 场景二：排除日志备份项目

```
tar -czvf project.tar.gz \
  --exclude="*.log" \
  --exclude="node_modules" \
  --exclude=".git" \
  /home/dev/project/
```

### 场景三：解压软件包到指定位置

```
# 下载的软件通常解压到 /opt 或 /usr/local
tar -xzvf nginx-1.24.0.tar.gz -C /opt/
```

## 动手练习

```
# 1. 创建测试目录
mkdir -p ~/test_compress/{docs,logs}
echo "hello" > ~/test_compress/docs/readme.txt
echo "error log" > ~/test_compress/logs/app.log

# 2. 打包压缩
tar -czvf test.tar.gz ~/test_compress/

# 3. 查看内容
tar -tvf test.tar.gz

# 4. 解压到 /tmp
tar -xzvf test.tar.gz -C /tmp/

# 5. 清理
rm -rf ~/test_compress test.tar.gz
```

## 2.8 综合实战练习

### 场景：模拟运维日常工作

你刚入职一家公司担任 Linux 运维工程师，需要完成以下任务：

### 任务清单

#### 任务一：创建项目目录结构

```
# 要求：创建以下目录结构
# /home/你的用户名/webapp/
# ├── config/
# ├── logs/
# ├── src/
# ├── backup/
# └── scripts/

# 你的操作：
cd ~
mkdir -p webapp/{config,logs,src,backup,scripts}
ls -la webapp/
```

## 任务二：创建配置文件

```
# 要求：在 config 目录下创建配置文件

cd ~/webapp/config
touch app.conf db.conf nginx.conf

# 添加一些内容
echo "# App Configuration" > app.conf
echo "port=8080" >> app.conf
echo "debug=true" >> app.conf

cat app.conf
```

## 任务三：备份配置文件

# 要求：将所有配置文件备份到 backup 目录

```
cp -r ~/webapp/config/* ~/webapp/backup/
```

# 给备份文件加上日期后缀

```
cd ~/webapp/backup
```

```
today=$(date +%Y%m%d)
```

```
for file in *.conf; do
```

```
    mv "$file" "${file%.conf}_${today}.conf"
```

```
done
```

```
ls -la
```

# 结果：app\_20241210.conf db\_20241210.conf nginx\_20241210.conf

## 任务四：创建常用链接

# 要求：在家目录创建快捷方式，方便访问

```
cd ~
```

```
ln -s ~/webapp/logs ~/logs
```

```
ln -s ~/webapp/config/app.conf ~/app.conf
```

```
ls -la | grep "^l"
```

```
# lrwxrwxrwx 1 tom tom ... logs -> /home/tom/webapp/logs
```

```
# lrwxrwxrwx 1 tom tom ... app.conf ->
```

```
/home/tom/webapp/config/app.conf
```

## 任务五：查找和清理临时文件

# 模拟创建一些临时文件

```
touch ~/webapp/logs/app.log
```

```
touch ~/webapp/logs/temp.log
```

```
touch ~/webapp/logs/debug.log
```

```
touch ~/webapp/src/main.py
```

```
touch ~/webapp/src/test_temp.py
```

# 要求1：找出所有日志文件

```
find ~/webapp -name "*.log"

# 要求2: 找出所有包含 temp 的文件
find ~/webapp -name "*temp*"

# 要求3: 查看日志文件内容 (假设已有内容)
tail -f ~/webapp/logs/app.log
# Ctrl+C 退出
```

## 任务六：整理和清理

```
# 要求: 删除测试用的临时文件和目录

# 先确认要删除的内容
ls ~/webapp/

# 交互式删除 (推荐)
rm -ri ~/webapp/

# 或者强制删除 (确认无误后)
# rm -rf ~/webapp/
```



## 课程总结

---

## 本课知识点回顾

| 类别     | 核心内容                                                                                              |
|--------|---------------------------------------------------------------------------------------------------|
| 文件系统概念 | 一切皆文件、目录树结构                                                                                       |
| 重要目录   | <code>/etc</code> （配置）、 <code>/var/log</code> （日志）、 <code>/home</code> （用户）                       |
| 路径     | 绝对路径从 <code>/</code> 开始，相对路径用 <code>.</code> <code>..</code> <code>~</code>                       |
| 目录操作   | <code>pwd</code> <code>cd</code> <code>ls</code> <code>mkdir</code> <code>rmdir</code>            |
| 文件操作   | <code>touch</code> <code>cp</code> <code>mv</code> <code>rm</code>                                |
| 内容查看   | <code>cat</code> <code>head</code> <code>tail</code> <code>less</code> ，重点记住 <code>tail -f</code> |
| 文件查找   | <code>find</code> （强大）、 <code>locate</code> （快速）、 <code>which</code> （命令）                         |
| 链接     | <code>ln</code> （硬链接）、 <code>ln -s</code> （软链接）                                                   |
| 打包压缩   | <code>tar</code> （打包）、 <code>zip/unzip</code> （压缩解压缩）                                             |